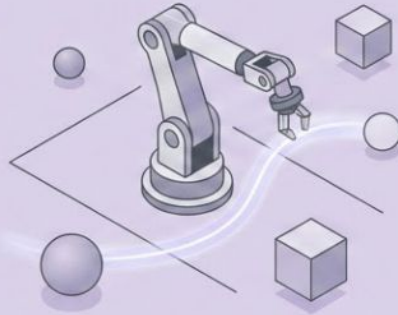


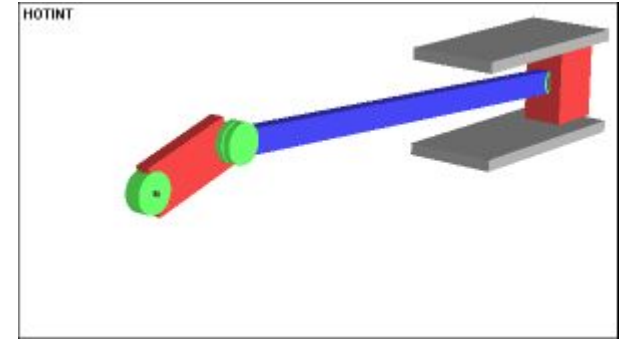
RRC Summer School



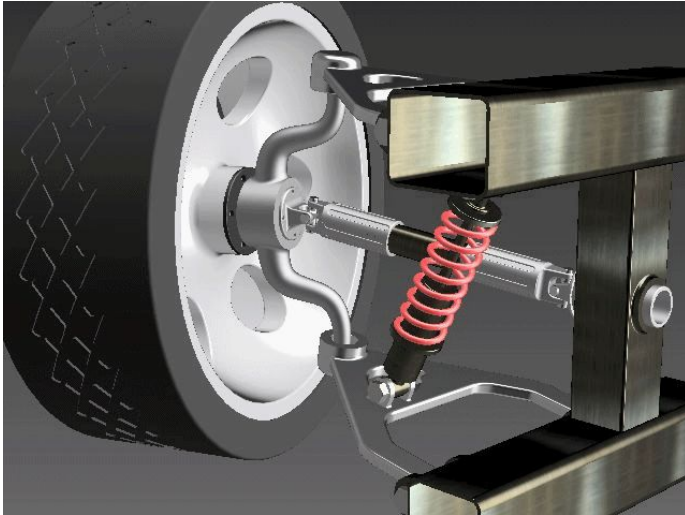
Motion Planning - IV
Robotic Arm Manipulation and Planning

Multi-Body Systems

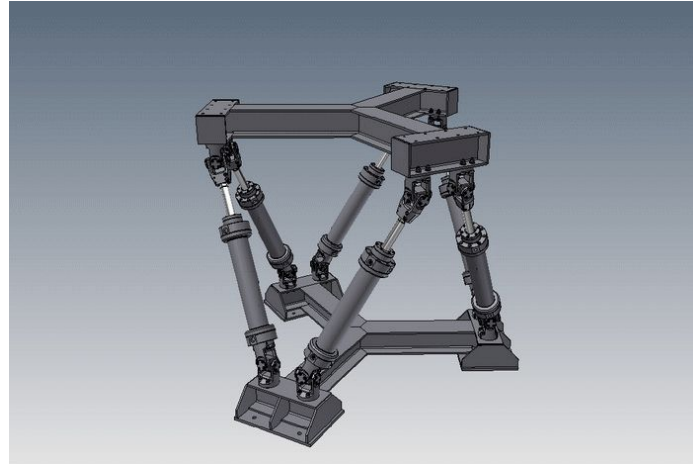
A multi-body system is a collection of several rigid body segments constrained together through interlinked joints



Slider-Crank Mechanism



Suspension Spring Mechanism

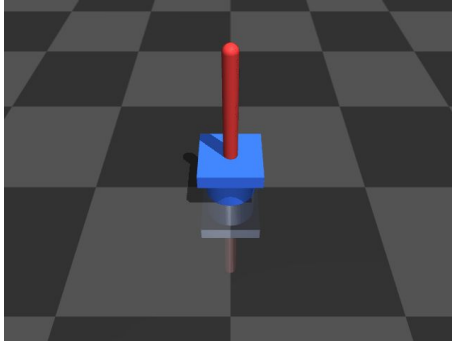


Parallel Manipulator

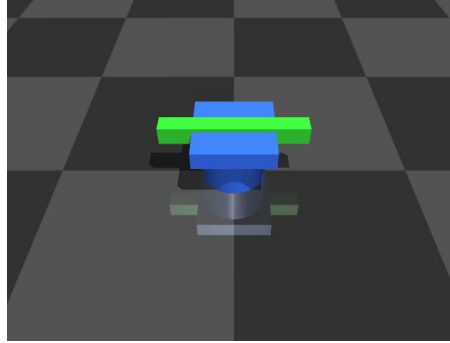


Serial Manipulator

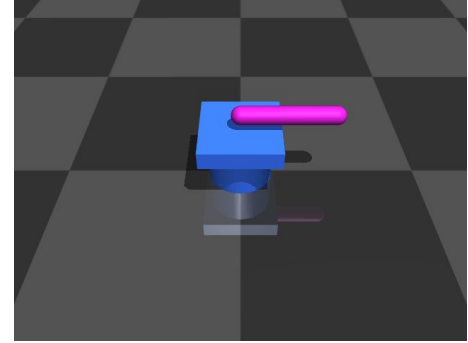
Common Types of Joints



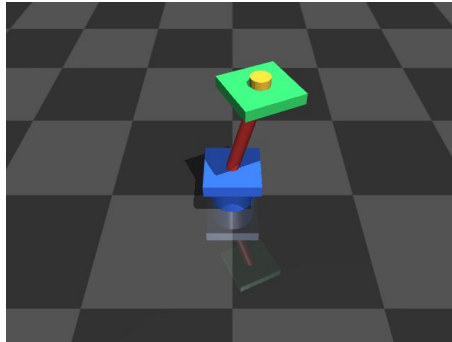
Revolute Joint



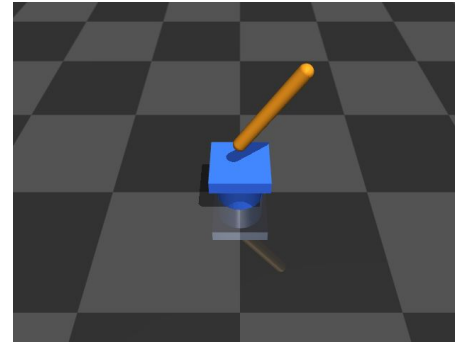
Prismatic Joint



Cylinder Joint



Universal Joint

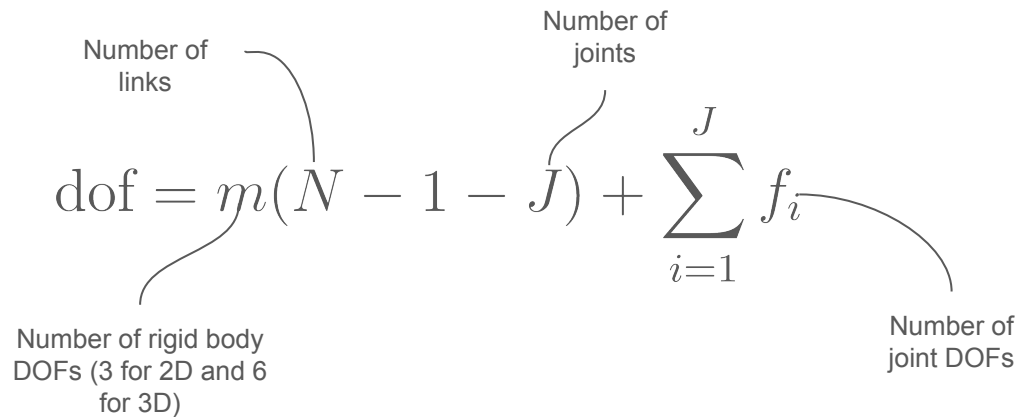


Spherical Joint

Degrees of Freedom

The minimum number of independent values required for representing the configuration of a given system is the number of **DOFs**.

Grübler's Formula



The diagram shows the equation for Grübler's Formula with three annotations. A bracket labeled 'Number of links' points to the variable N . A bracket labeled 'Number of joints' points to the variable J . A bracket labeled 'Number of joint DOFs' points to the summation term $\sum_{i=1}^J f_i$. A bracket labeled 'Number of rigid body DOFs (3 for 2D and 6 for 3D)' points to the multiplier m .

$$\text{dof} = m(N - 1 - J) + \sum_{i=1}^J f_i$$

Number of links

Number of joints

Number of joint DOFs

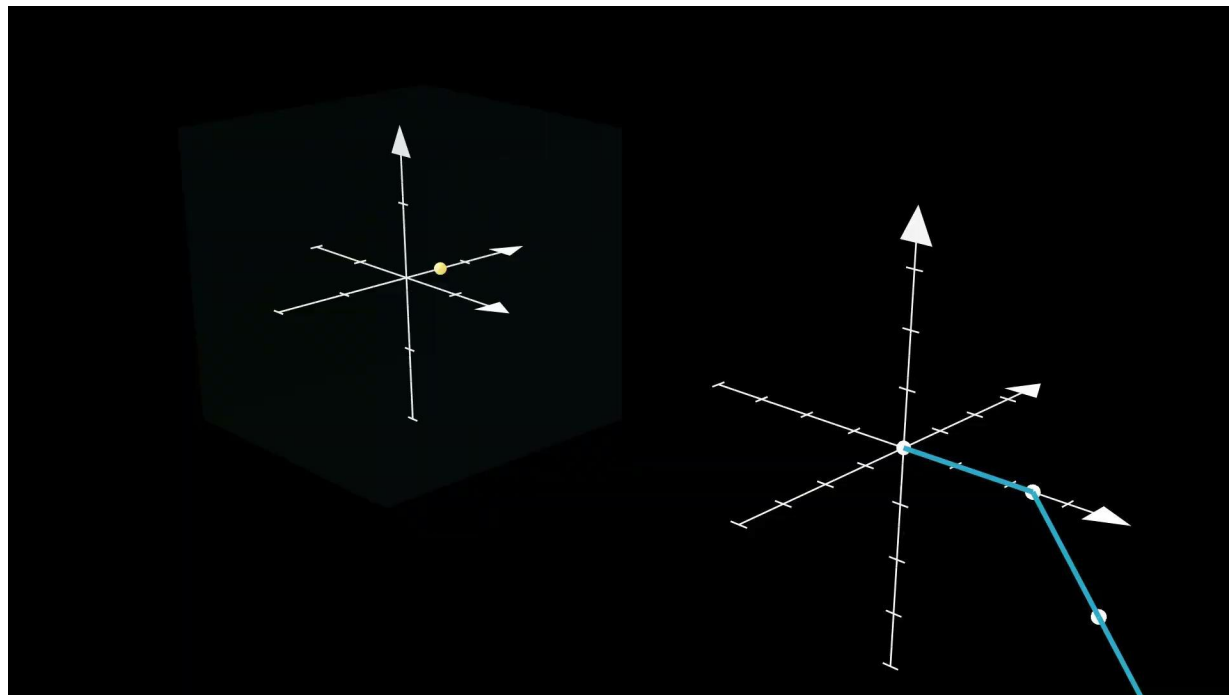
Number of rigid body DOFs (3 for 2D and 6 for 3D)

Forward Kinematics

A mapping from the configuration space of the robot to the workspace which gives the position of the end-effector

$$f : \mathcal{C} \rightarrow SE(2)$$

$$f : \mathcal{C} \rightarrow SE(3)$$

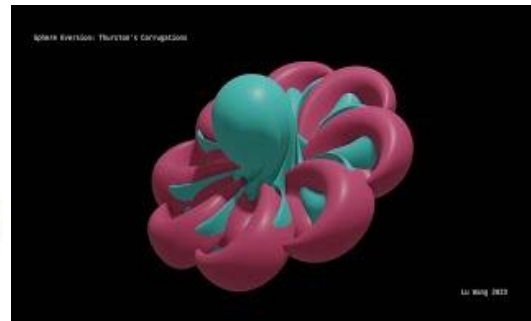
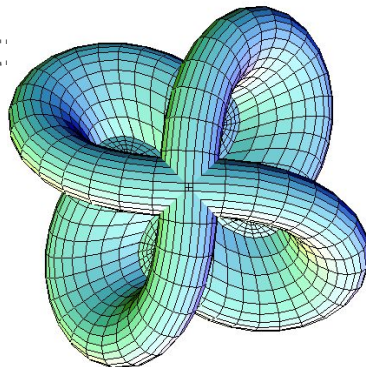


Configuration Space

A Manifold is a topological space such that:

- It is a Hausdorff Space
- It is second-countable
- It is locally Euclidean

If is smooth then we can do calculus on it



A group is a set G equipped with a binary operator $*$ with the following properties:

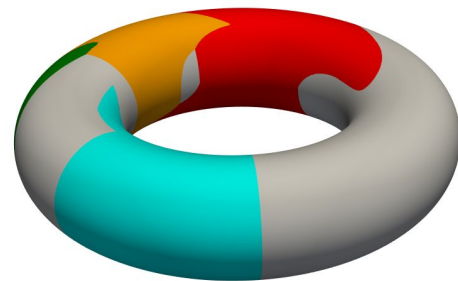
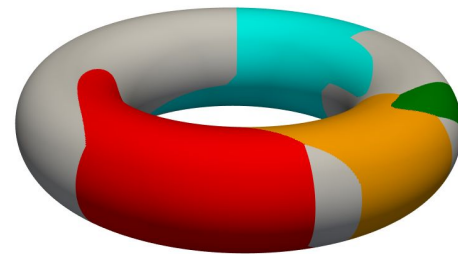
- Associativity : $a * (b * c) = (a * b) * c$
- Closure : if $c = a * b$ then $c \in G$
- Identity : $e \in G$ s.t. $a * e = e * a = a$
- Inverse : if $a \in G$ then $\exists b \in G$ s.t. $a * b = b * a = e$

Configuration Space

Combining these two mathematical tools gives us the almighty “Lie Groups”

Examples: $SO(2)$, $SO(3)$, $SE(2)$, $SE(3)$, $GL(2)$, etc.

The Lie Group formation allows use to define the tangent space of the manifold called Lie Algebra that lets us to compute derivatives for State Estimation as well to understand the velocity space



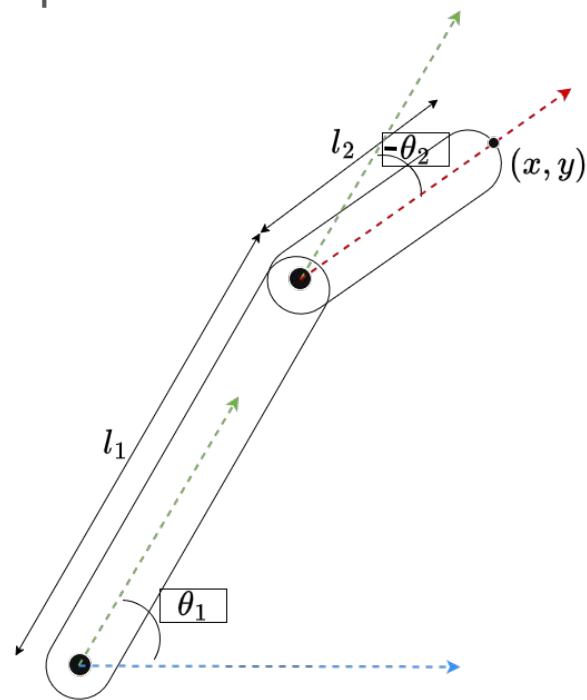
Forward Kinematics

Forward kinematics for 2R-manipulator in a 2D workspace

$$f : \mathcal{C} \rightarrow \mathbb{R}^2$$

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

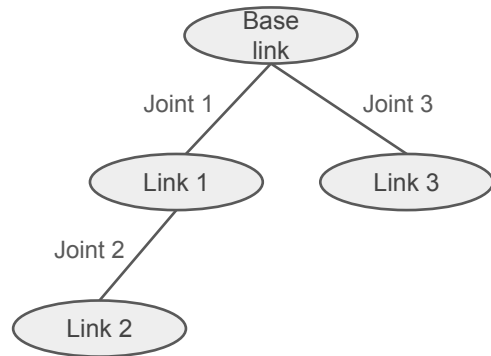
$$y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$



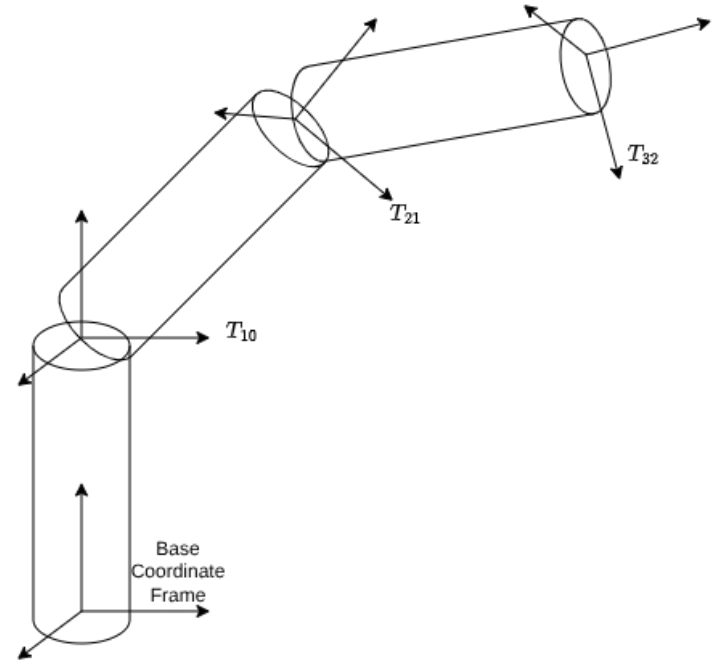
Forward Kinematics

$$T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}$$

$$T_{ee} = T_{32}T_{21}T_{10}$$



Kinematic/TF Tree



Forward Kinematics

Denavit-Hartenberg Frame assignment

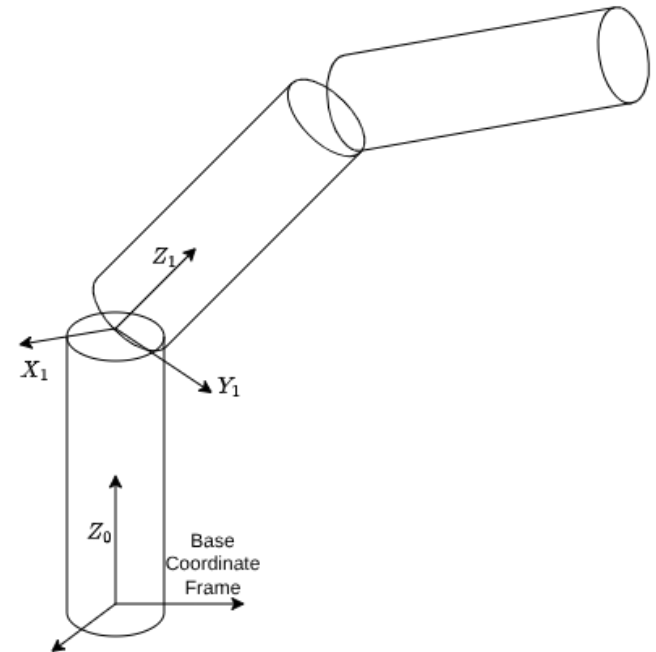
Assign a Base Coordinate Frame (X_0, Y_0, Z_0) such that is parallel to axis of motion of joint 1.

Draw Z_i along the axis of motion of joint $i+1$

The origin of the coordinate frame is the intersection of Z_i and Z_{i-1} or the common normal between Z_i & Z_{i-1}

Draw X_i along the cross product of Z_i and Z_{i-1} or along the common normal Z_i & Z_{i-1}

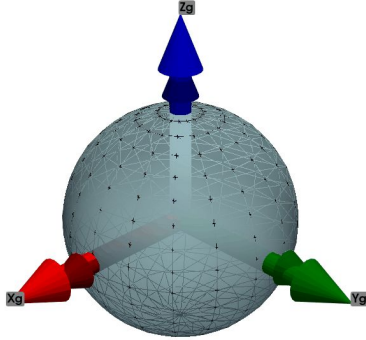
Draw Y_i axis to form an orthogonal coordinate frame



Forward Kinematics

Euler Angle Representation

Intrinsic ZYX Euler Rotation
psi = 1.5°
theta = 0.0°
phi = 0.0°



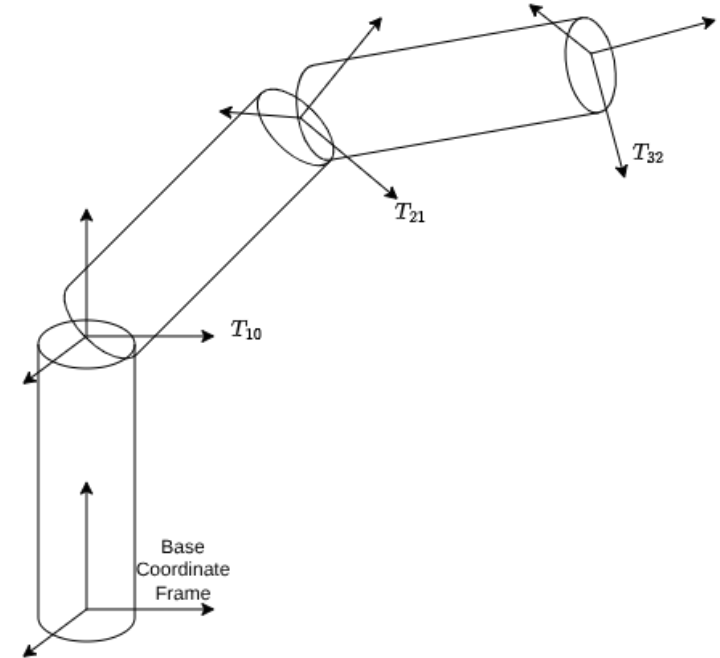
$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$[R = R_z(\psi)R_y(\theta)R_x(\phi)]$$

$$R = \begin{bmatrix} \cos\psi\cos\theta & \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi & \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \sin\psi\cos\theta & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{bmatrix}$$



Forward Kinematics

Quaternion Representation

Rodrigues Formula:

$$\vec{p}_{rot} = \cos(\theta)\vec{p} + \sin(\theta)\vec{n} \times \vec{p} + (1 - \cos \theta)(\vec{n} \cdot \vec{p})\vec{n}$$

Quaternion:

$$q = q_w + q_x i + q_y j + q_z k \text{ where } i^2 = j^2 = k^2 = ijk = -1$$

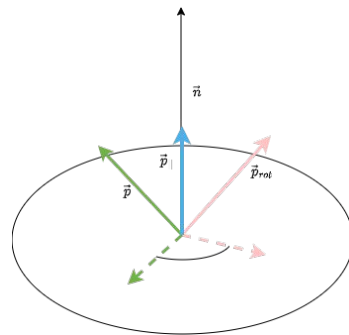
$$\text{Cuter version: } q = q_w + \vec{q}$$

Quaternion Multiplication:

$$pq = p_w q_w - \vec{p} \cdot \vec{q} + p_w \vec{q} + q_w \vec{p} + \vec{p} \times \vec{q}$$

Quaternion Rotation:

$$\vec{p}_{rot} = q\vec{p}q^* \text{ where } q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)\vec{n}$$



Tomb Raider (1996)

Forward Kinematics

Product of exponentials

$$R(\theta) = e^{\hat{\omega}\theta}$$

$$T(\theta) = e^{\hat{e}\theta}$$

$$\epsilon = \begin{bmatrix} v \\ \omega \end{bmatrix} \quad \text{where} \quad \epsilon \in SE(3)$$

$$T(\theta) = e^{\epsilon_1 \theta_1} e^{\epsilon_2 \theta_2} \dots e^{\epsilon_n \theta_n} M$$

Forward Kinematics

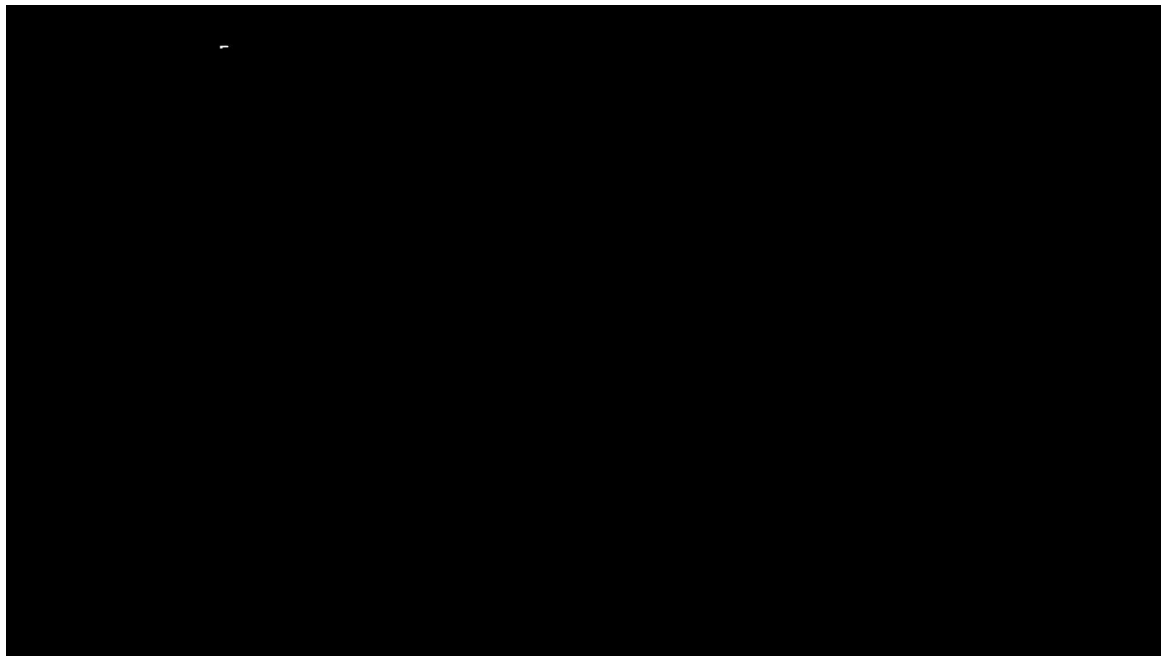
The forward kinematics is a non-linear mapping but is locally linear

Noob-level approximation

$$f(q) \approx f(q_0) + f'(q_0)(q - q_0)$$

Chad-level approximation

$$f(\mathbf{q}) \approx f(\mathbf{q}_0) + J(\mathbf{q}_0)(\mathbf{q} - \mathbf{q}_0)$$



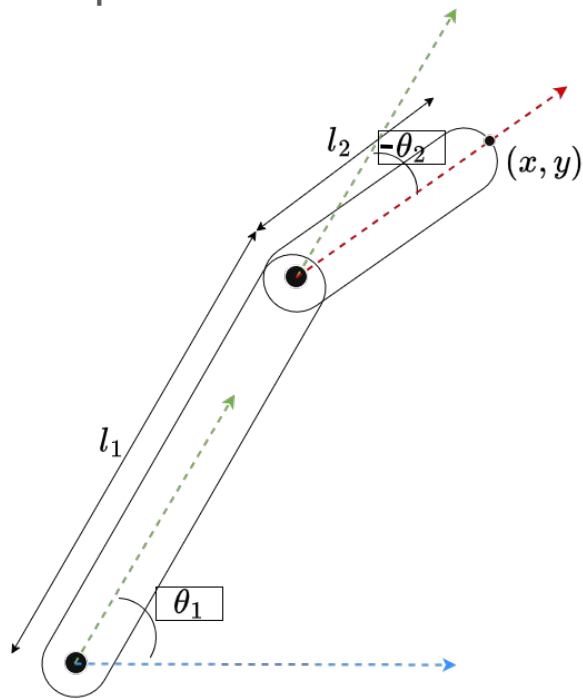
Inverse Kinematics

Mapping back from the workspace to the configuration space

$$\cos \theta_2 = \frac{x^2 + y^2 - l_1^2 - l_2^2}{2l_1l_2}$$

$$\theta_2 = \text{atan2}(\pm\sqrt{1 - \cos^2(\theta_2)}, \cos(\theta_2))$$

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(l_2 \sin(\theta_2), l_1 + l_2 \cos(\theta_2))$$



Inverse Kinematics

$$\min_q |f(q) - \vec{x}|$$

Implicit Function Theorem:

Any implicit function can be represented as an explicit function locally using the Jacobian of the entire function

Differential Kinematics:

$$q_{k+1} = q_k + (J^T J)^{-1} J^T (f(q_k) - \vec{x})$$

Inverse Kinematics

Kids use:

- Newton (second order)
- Gauss-Newton

Adults use:

- Sequential Quadratic Programming
- Interior Point Methods
- Augmented Lagrangian Methods
- Trust region based Newton optimization

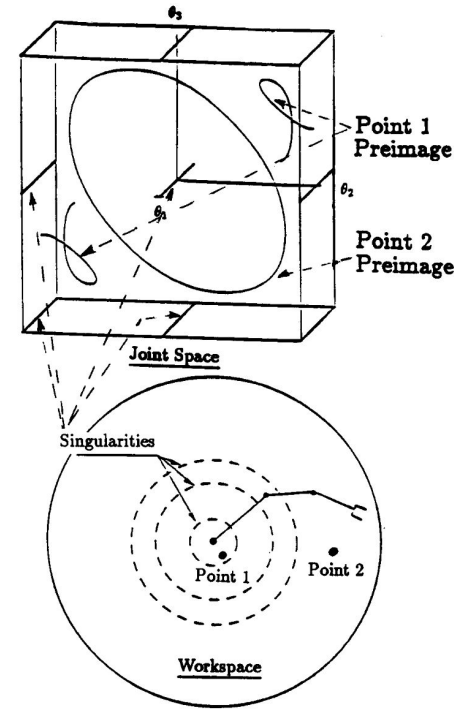
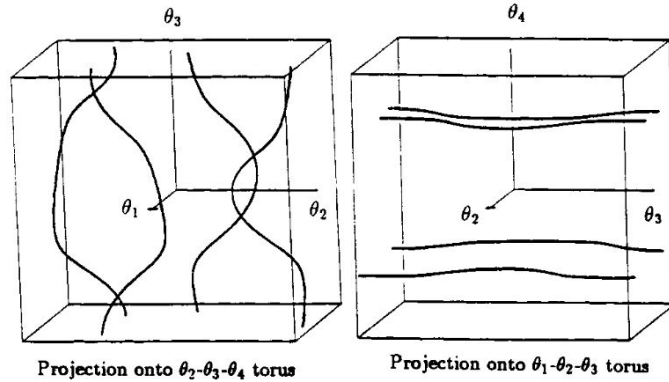
Le Legends:



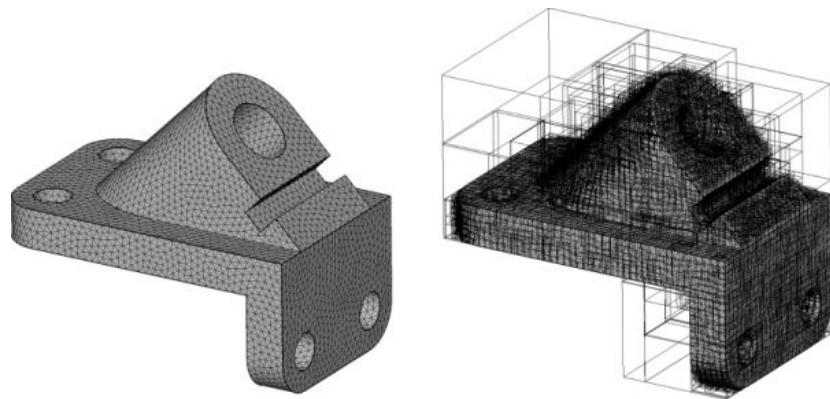
Imma do it
by hand

Inverse Kinematics

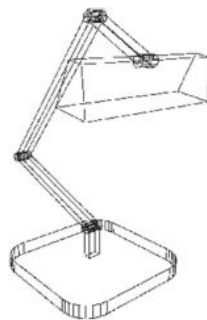
C-space Bundles and W-sheets



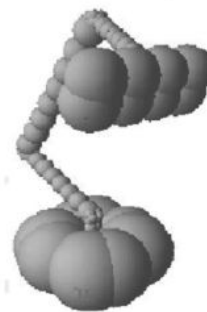
Obstacle Representation



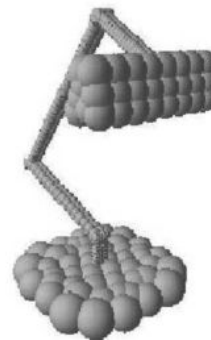
Axis aligned bounding boxes



(a) Non-Convex
Object: Lamp



(b) Level 3



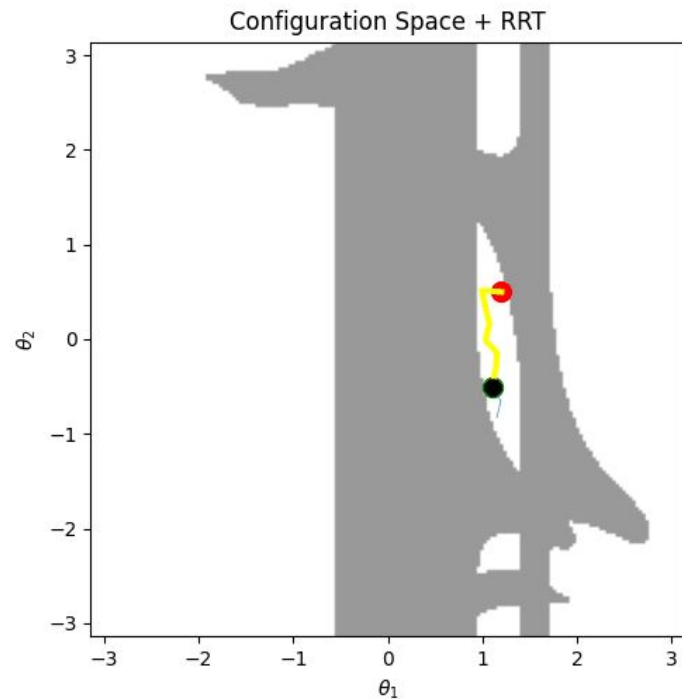
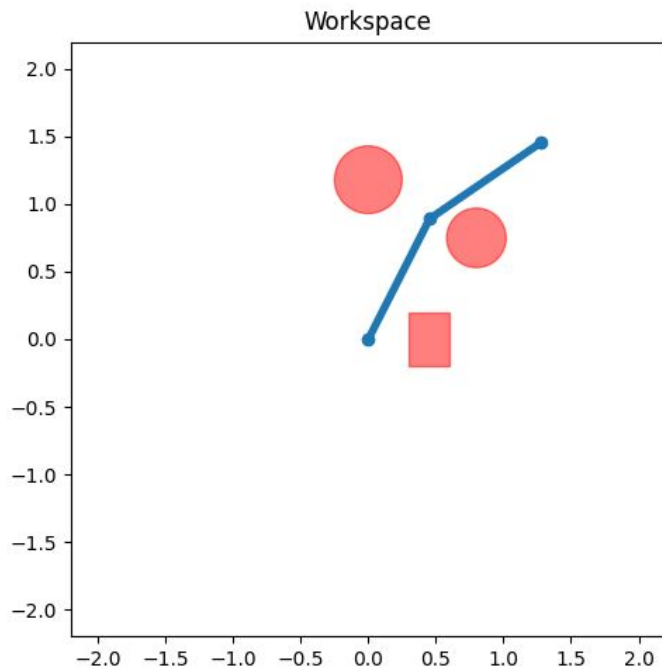
(c) Level 6



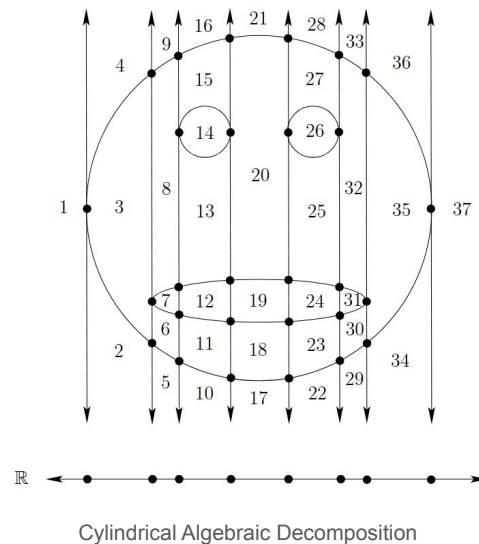
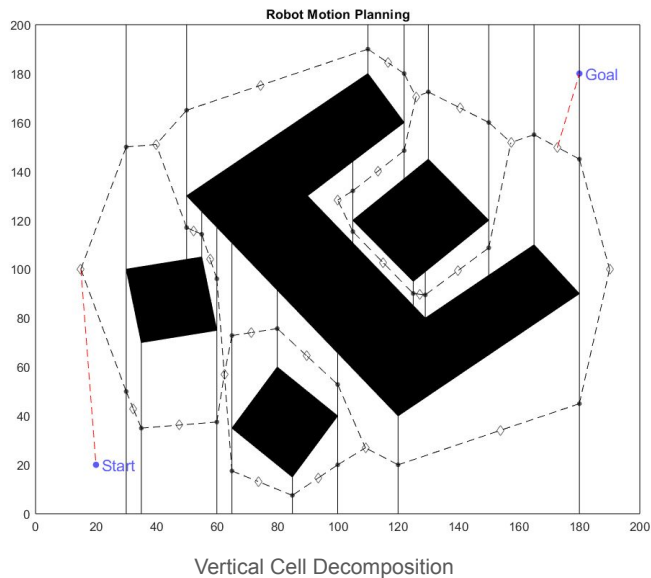
(d) Level 12

Spheres for BVH

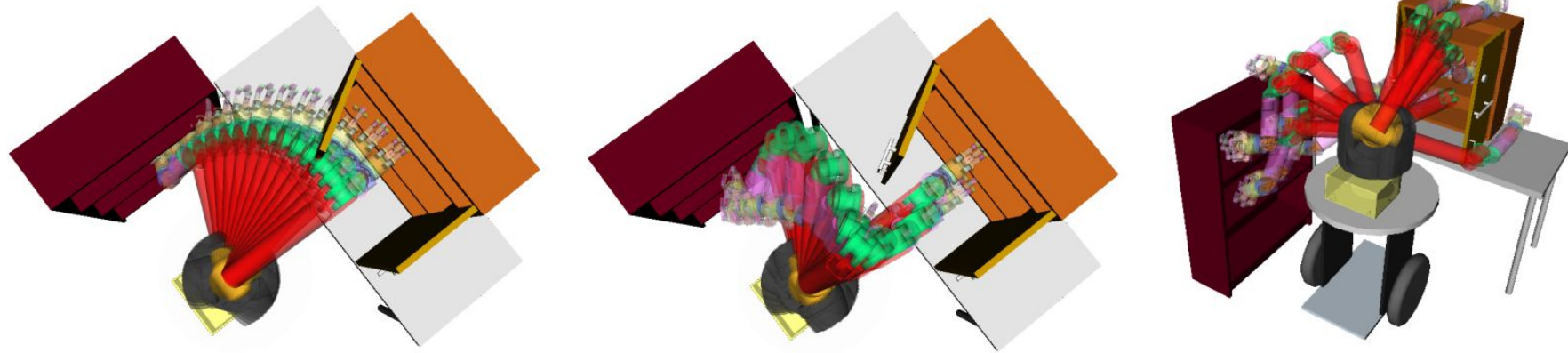
Motion Planning in configuration space



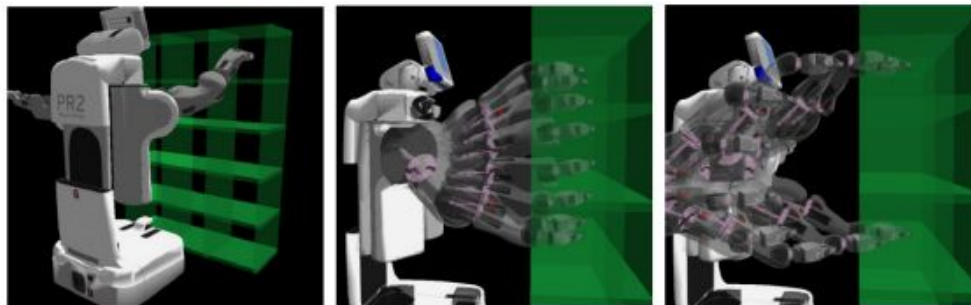
Complete Motion Planning



Optimal Planning



Covariant Hamiltonian Optimization for Motion Planning (CHOMP)



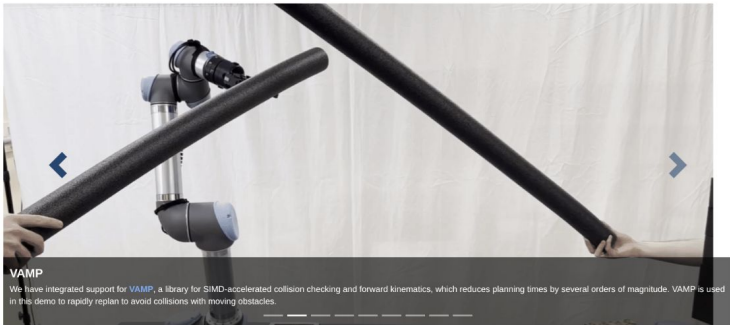
Stochastic Trajectory Optimization for Motion Planning (STOMP)

Motion Planning in configuration space

Open Motion Planning Library (OMPL)

[OMPL](#) [Download](#) [Documentation](#) [Gallery](#) [OMPL Integrations](#) [Code](#) [Issues](#) [Community](#) [About](#) [Blog](#)

The Open Motion Planning Library



VAMP
We have integrated support for **VAMP**, a library for SIMD-accelerated collision checking and forward kinematics, which reduces planning times by several orders of magnitude. VAMP is used in this demo to rapidly replan to avoid collisions with moving obstacles.

OMPL, the Open Motion Planning Library, consists of many state-of-the-art sampling-based motion planning algorithms. OMPL itself does not contain any code related to, e.g., collision checking or visualization. This is a deliberate design choice, so that OMPL is not tied to a particular collision checker or visualization front end. The library is designed so it can be easily integrated into [systems that provide the additional needed components](#). OMPL has been used to plan motions for a broad range of robotic systems including: (mobile) manipulators, car-like systems, drones, and underwater vehicles.

OMPL includes complete examples on how to integrate with libraries that perform collision checking, kinematics, model loading, etc. By default, support for VAMP is enabled. **VAMP** is a library for SIMD-accelerated collision checking and forward kinematics, which reduces planning times by several orders of magnitude. We have also included support for **Pinocchio**, a library for forward/inverse kinematics and dynamics.

OMPL aims to make it straightforward to evaluate new planning algorithms on realistic robots in realistic settings through integrations with libraries like VAMP and Pinocchio. Developers of new algorithms are encouraged to use OMPL's [benchmarking capabilities](#). Benchmark results can be explored interactively in [Planner Arena](#), an external package like [Planner Developer Tools](#) or your own plotting tool!

[Download version 2.0.0](#)
Released Apr 06, 2019

[Click for citation](#)
If you use OMPL in your work

Library Contents

- OMPL contains implementations of many sampling-based algorithms such as PRM, RRT, EST, SBL, KPIECE, SyCLOR, and several variants of these planners. See [available planners](#) for a complete list.
- All these planners operate on very abstractly defined state spaces. Many commonly used state spaces are already implemented (e.g., SE(2), SE(3), Rⁿ).

Getting Started

- The **OMPL primer** provides a brief background on sampling-based motion planning, and an overview of OMPL.
- [Download and install OMPL](#).
- [Demos and tutorials](#).
- [Frequently Asked Questions](#).
- [Learn how to integrate your own code with OMPL's](#)

Other Resources

- [Gallery of example uses of OMPL](#).
- If you use ROS, the recommended way to use OMPL is through [MoveIt](#).
- [Third-party contributions](#). (Contribute your own extensions!)

Motion Planning in configuration space

Open Motion Planning Library (OMPL)

```
auto space = std::make_shared<ob::RealVectorStateSpace>(ndof);
auto si = std::make_shared<ob::SpaceInformation>(space);
si->setStateValidityChecker(std::make_shared<StateValidator>(si));

auto start_state = si->allocState();
auto goal_state = si->allocState();
{
    auto* sr = start_state->as<ob::RealVectorStateSpace::StateType>();
    auto* gr = goal_state->as<ob::RealVectorStateSpace::StateType>();
    for (int i = 0; i < ndof; ++i) {
        sr->values[i] = q_start[i];
        gr->values[i] = q_goal[i];
    }
}

og::PRMGraph prm(si, q_start, q_goal);
```


Motion Planning in configuration space

Open Motion Planning Library (OMPL)

```
auto pdef = std::make_shared<ob::ProblemDefinition>(si);
pdef->setStartAndGoalStates(start_state, goal_state);
prm.setProblemDefinition(pdef);
prm.setup();

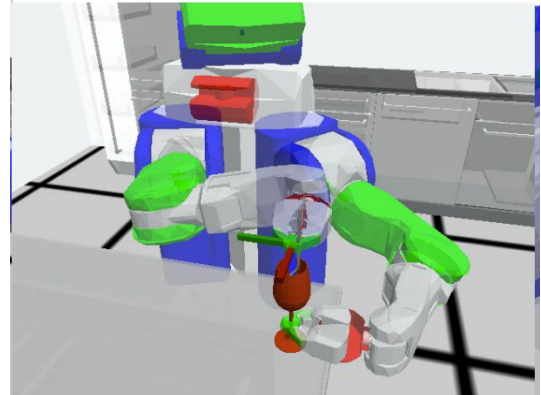
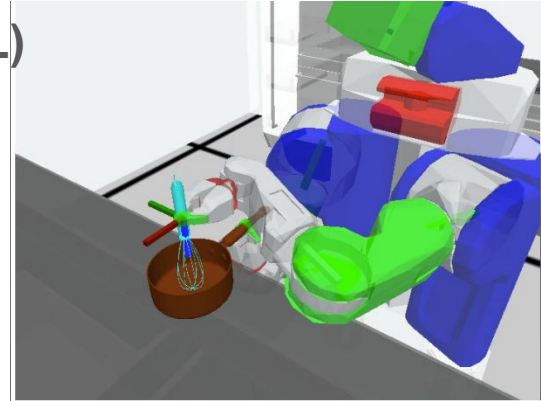
prm.growRoadmap(ptc);
prm.expandRoadmap(ptc);

ob::PlannerStatus solved = prm.solve(ob::timedPlannerTerminationCondition(5.0));
```

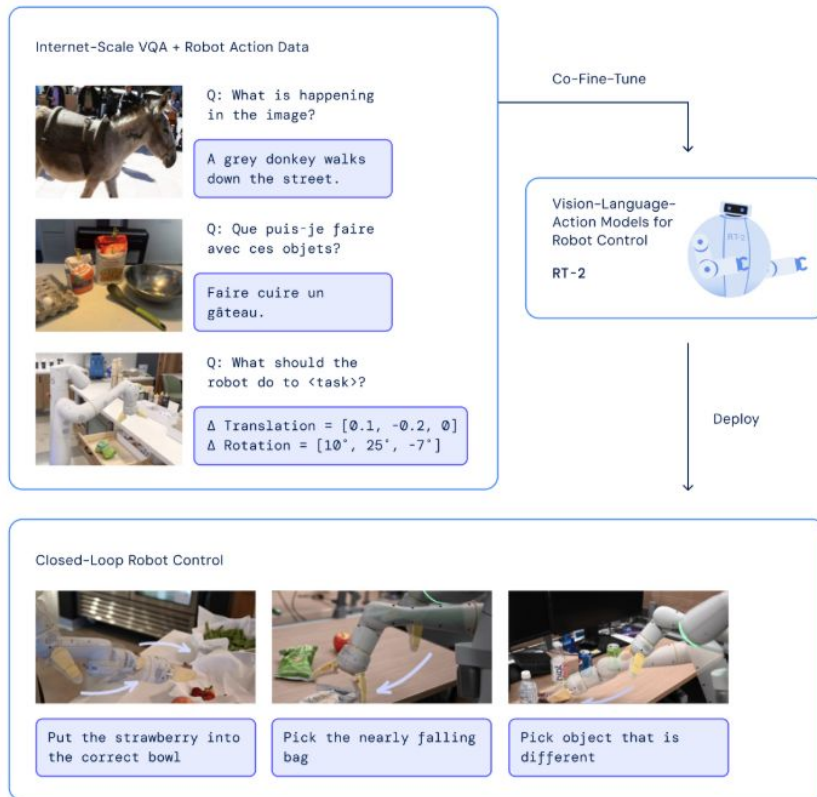
Task Planning (TAMP)

Planning Domain Definition Language (PDDL)

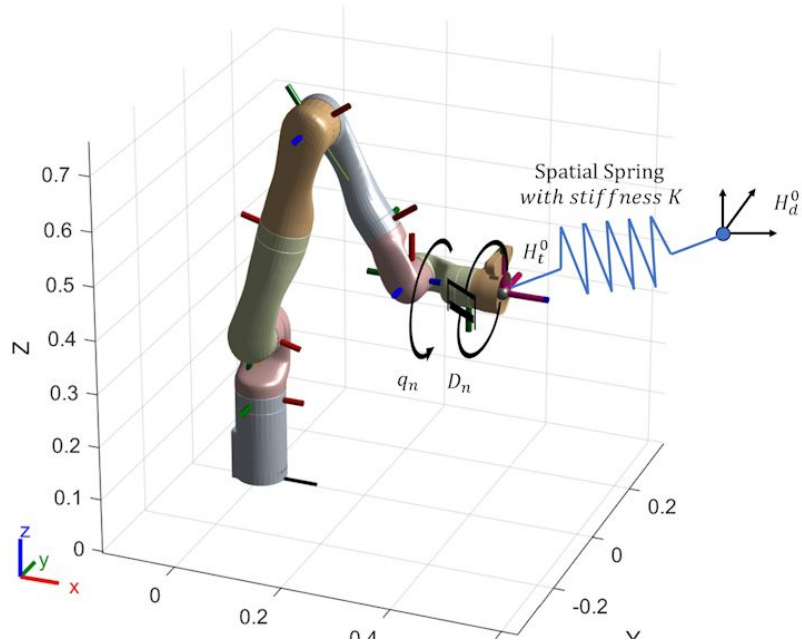
```
(define (problem vehicle-example)
  (:domain vehicle)
  (:objects
    truck car - vehicle
    full half empty - fuel-level
    Paris Berlin Rome Madrid - location)
  (:init
    (at truck Rome)
    (at car Paris)
    (fuel truck half)
    (fuel car full)
    (next full half)
    (next half empty)
    (accessible car Paris Berlin)
    (accessible car Berlin Rome)
    (accessible car Rome Madrid)
    (accessible truck Rome Paris)
    (accessible truck Rome Berlin)
    (accessible truck Berlin Paris)
  )
  (:goal (and (at truck Paris)
              (at car Rome)))
)
```



Vision and Language Action Models



Manipulator Control



References

Books:

- [MODERN ROBOTICS: MECHANICS, PLANNING, AND CONTROL](#)
- [Planning Algorithms \(Steven Lavalle\)](#)
- [Robotics: Modelling, Planning and Control](#)

Paper:

- [A micro Lie theory for state estimation in robotics](#)
- [PDDL Stream: Integrating Symbolic Planners and Blackbox Samplers via Optimistic Adaptive Planning](#)

Links:

- [Robotic Manipulation \(Russ Tedrake\)](#)
- [Introduction to Robotics \(Oussama Khatib\)](#) - playlist